

MSC ARTIFICIAL INTELLIGENCE
MASTER THESIS

Reducing And-Inverter Graphs for Scalable Optimizability Prediction

by
Isabella V. GARDNER
314159265

2026/01/05–2026/08/31
36 ECTS

Supervisor: Michael COCHEZ

Examiner: Pascal METTES



UNIVERSITEIT VAN AMSTERDAM
Instituut voor Informatica

Reducing And-Inverter Graphs for Scalable Optimizability Prediction

This work by Isabella V. GARDNER is licensed under **CC-BY-NC**.

<https://creativecommons.org/licenses/by-nc/4.0>



This license requires that reusers give credit to the creator. It allows reusers to distribute, remix, adapt, and build upon the material in any medium or format, for noncommercial purposes only.

Abstract

Contents

List of Figures	vii
List of Tables	ix
List of Symbols	xi
List of Abbreviations	xiii
1 Introduction	1
1.1 Motivation	1
1.1.1 Logic Synthesis and Optimization Script Design	1
1.1.2 Machine Learning for AIGs	1
1.1.3 The Scale Bottleneck	1
1.1.4 Limitations of Generic Graph Reduction	1
1.1.5 Problem Statement and Research Gap	2
1.2 Research Questions	2
1.2.1 RQ1: Task Feasibility and Baseline	2
1.2.2 RQ2: Reduction Efficiency	2
1.2.3 RQ3: Predictive Retention and Trade-offs	2
1.2.4 RQ4: Value of Domain-Informed Adaptation	2
1.2.5 RQ5: Cross-State Inference	2
1.2.6 Scope and Delimitations	2
1.2.7 Problem Formalization	3
1.3 Contributions	3
1.4 Thesis Outline	3
2 Preliminaries & Related Work	5
2.1 Preliminaries	5
2.1.1 Electronic Design Automation and Logic Synthesis	5
2.1.2 And-Inverter Graphs	5
2.1.3 Synthesis Algorithms	5
2.1.4 Graph Neural Networks	5
2.1.5 Graph Reduction: Definitions	5
2.2 Related Work	5
2.2.1 Machine Learning for Logic Synthesis	5
2.2.2 Scaling Graph Neural Networks to Large Graphs	6
2.2.3 Graph Reduction Techniques	6
2.2.4 Structure-Preserving and Domain-Aware Reduction	6
2.2.5 Size Generalization and Cross-State Transfer	6
2.2.6 Research Gap	6
3 Methodology	7
3.1 Architecture	7
3.1.1 Encoder Formulation	7
3.1.2 Partitioned Inputs	7
3.1.3 Sparsified Inputs	7
3.1.4 Summarized Inputs	7
3.1.5 Encoder Variant for Exact Colour Refinement	7
3.1.6 Baselines for Optimizability Prediction	7
3.2 Graph Reduction	7
3.2.1 Partitioning	7
3.2.2 Sparsification	7
3.2.3 Summarization and Coarsening	7

3.3	Data	8
3.3.1	Source Circuits	8
3.3.2	Dataset Generation	8
3.3.3	Tiered Dataset Structure	8
3.3.4	Target Label: Optimizability	8
3.3.5	Graph Representation	8
3.3.6	Dataset Analysis	8
3.3.7	Held-Out Hyperparameter Tuning Subset	8
3.4	Experiments	8
3.4.1	Research Design	8
3.4.2	Setup	8
3.4.3	Evaluation Metrics	8
3.4.4	Reproducibility	8
4	Results	9
4.1	Baseline Predictive Performance on Full AIGs (RQ1)	9
4.1.1	Comparison Against Naive Baselines	9
4.1.2	Predictive Accuracy Metrics	9
4.1.3	Ranking Efficacy	9
4.1.4	Baseline Model Outcomes	9
4.1.5	Error Analysis	9
4.1.6	Protocol Sensitivity: How Much of the Score Is Design Recognition? (RQ1a)	9
4.1.7	Cost of the Full-Graph Baseline	9
4.2	Efficiency Profile of Graph Reduction Techniques (RQ2)	9
4.2.1	Graph Reduction Offline Profile	9
4.2.2	Training Memory Dynamics and Compute Efficiency	9
4.2.3	Throughput and Training Speedup	9
4.2.4	Efficiency Ranking Across Methods	9
4.3	Predictive Retention and Performance Trade-offs (RQ3)	9
4.3.1	Accuracy Degradation	9
4.3.2	Ranking Preservation	9
4.3.3	Accuracy Loss Attributable to Reduction	9
4.3.4	The Pareto Front (Trade-Off Analysis)	9
4.4	Value of Domain-Informed Adaptation (RQ4)	9
4.4.1	Matched-Compression Pairings	9
4.4.2	Retention Gap at Matched Compression	9
4.4.3	Structural Interpretation	9
4.4.4	Adequacy of the Heuristics Tested	9
4.5	Cross-State Inference (RQ5)	9
4.5.1	Reduced-to-Full Inference Accuracy	9
4.5.2	Zero-Shot Ranking	9
4.5.3	Matched-State against Cross-State Drop-Off	9
4.5.4	Cross-State Transfer by Reduction Family	9
4.5.5	Inference Cost	9
4.6	Summary of Findings	9
4.6.1	Consolidated Summary	9
5	Discussion	11
5.1	Comparison with Related Work	11
5.1.1	Positioning Against Optimizability Prediction Work	11
5.1.2	Positioning Against Graph Reduction Work	11
5.1.3	Where Direct Comparison Is Not Possible	11
5.2	Interpretation of Findings	11
5.2.1	Feasibility of Optimizability Prediction	11
5.2.2	What Reduction Buys and What It Costs	11
5.2.3	Why Domain-Informed Adaptation Did or Did Not Help	11
5.2.4	Cross-State Transfer and Deployment	11
5.2.5	Unexpected Results	11
5.3	Practical Implications	11
5.4	Limitations	11
5.4.1	Reproducibility	11

5.4.2	Scalability	11
5.4.3	Generalizability	11
5.4.4	Reliability and Validity	11
6	Conclusion	13
6.1	Answers to the Research Questions	13
6.1.1	RQ1: Task Feasibility and Baseline	13
6.1.2	RQ2: Reduction Efficiency	13
6.1.3	RQ3: Predictive Retention and Trade-offs	13
6.1.4	RQ4: Value of Domain-Informed Adaptation	13
6.1.5	RQ5: Cross-State Transfer	13
6.2	Contributions Revisited	13
6.3	Future Work	13
6.3.1	Extending Beyond a Single Synthesis Algorithm	13
6.3.2	Stronger Domain-Aware Reduction	13
6.3.3	Scaling to Industrial AIGs	13
6.3.4	From Prediction to Script Generation	13
A	First Appendix	17
A.1	Synthesis Algorithm Configurations	17
A.2	Supplementary Figures	17
A.3	Experimental Configuration	17
A.4	Hyperparameter Search Space	17
A.5	Library and Tool Versions	17
A.6	Reduction Method Parameters	17
A.7	Baseline Hyperparameters	17
A.8	Extended Results	17

List of Figures

List of Tables

List of Symbols

Symbol	Description
Graph and structural notation	
$G = (\mathcal{V}, \mathcal{E})$	And-Inverter Graph (AIG): a directed acyclic graph over the basis $\{\wedge, \neg\}$, with node (gate) set $\mathcal{V}$ and edge set $\mathcal{E}$ (2.1.2).
τ	Node-role labelling, $\tau: \mathcal{V} \rightarrow \{\text{const}, \text{PI}, \text{AND}, \text{PO}\}$ (??).
inv	Edge-inversion tag, $\text{inv}: \mathcal{E} \rightarrow \{0, 1\}$, marking a connection as regular or inverted (??).
$\text{fi}(v), \text{fo}(v)$	Fanin set and fanout set of node v (??).
$\ell(v), \ell(G)$	Topological level of node v , and maximum level of G : $\ell(G) = \max_{v \in \mathcal{V}} \ell(v)$ (??).
$\mathcal{C}(v), \mathcal{F}(v)$	Fanin (causal) cone and fanout cone of node v (??).
$\text{MFFC}(v)$	Maximal Fanout-Free Cone of v : the gates that feed v exclusively, removed with v under restructuring (??).
Task, model, and reduction operators	
S	Deterministic synthesis algorithm (Orchestrate); $S(G)$ is the function-equivalent, optimized circuit (2.1.3).
$y(G)$	Optimizability of G under S : the fraction of nodes S removes, $y(G) = 1 - \mathcal{V}(S(G)) / \mathcal{V}(G) \in [0, 1]$ (1.3).
f_θ	Trained GNN encoder, approximating y from an input AIG (1.2.7).
L	Number of message-passing layers in the GNN, i.e. the radius of a node's receptive field (2.1.4).
R	Reduction operator; $R(G)$ is the reduced graph, and $R = \text{id}$ denotes the unreduced baseline (2.1.5).
d	Colour-refinement depth: the number of refinement rounds run when building a reduct. Distinct from the topological level ℓ ; exactness requires $L \leq d$ (2.1.5).
Measuring reduction	
$\eta_{\mathcal{V}}(G), \eta_{\mathcal{E}}(G)$	Node and edge retention (compression) ratio of a reduction: $ \mathcal{V}(R(G)) / \mathcal{V}(G) $ and $ \mathcal{E}(R(G)) / \mathcal{E}(G) $ (??).
n^*	Amortisation threshold: the number of epochs beyond which applying a reduction is cheaper than training on the unreduced graph (??).

List of Abbreviations

Abbreviation	Expansion
AIG	And-Inverter Graph
EDA	Electronic Design Automation
FRAIG	Functionally Reduced AIG
GNN	Graph Neural Network
LS	Logic Synthesis
MFFC	Maximal Fanout-Free Cone
PI	Primary Input
PO	Primary Output

Introduction

1.1 Motivation

1.1.1 Logic Synthesis and Optimization Script Design

Electronic Design Automation (EDA) supplies the tools that convert Boolean specifications into the integrated circuits underlying nearly every computing device modern life depends on, from consumer electronics to the data centers and communication networks behind it. Logic synthesis and optimization remain the foundational steps within EDA for translating high-level Boolean specifications into efficient, manufacturable hardware. One of the central challenges of this field is efficiently representing and manipulating Boolean functions, particularly as the complexity of digital circuits increases. Traditional methods such as truth tables, sum-of-products (SOP), or binary-decision diagrams (BDD) became impractical for large-scale designs due to their potential exponential growth in size. And-Inverter Graphs (AIGs) address this issue by providing a more scalable and compact data structure for Boolean function representation [Kuehlmann et al., 2002], and are the state-of-the-art for technology-independent logic optimization, supporting a wide range of synthesis and optimization tasks fundamental to EDA [Mishchenko et al., 2006, Brayton and Mishchenko, 2010]. Over the past few decades, numerous optimization algorithms have been developed for AIGs, each targeting a different objective, among them node count, logic depth, area, delay, and power consumption [Mishchenko and Brayton, 2006].

These algorithms are applied as a sequence, and their order is itself a determinant of the outcome: the same set of algorithms yields different results depending on the order in which they are applied. Crafting an effective sequence, a synthesis *script*, for a given circuit is therefore a heuristic trial-and-error process: each candidate script must run to completion before its result is known.

1.1.2 Machine Learning for AIGs

Machine Learning for Logic Synthesis

Structural representations such as AIGs provide a foundation for applying machine learning (ML) to address a multitude of complex challenges in logic synthesis. These methods divide into two classes: those that predict a synthesis outcome, and those that replace the heuristic that produces it [Liu et al., 2026]. Progress on both fronts remains constrained by the absence of standardized, large-scale AIG datasets and benchmarks [Chowdhury et al., 2023].

Predicting Optimizability

A model that predicts a circuit’s *optimizability*, the fraction of nodes a given script removes from it, would circumvent that trial-and-error search by supporting data-driven script generation directly, and, arguably, by revealing how much a circuit can shrink before an expensive optimization flow completes. *Graph Neural Networks* (GNNs) are the natural class of models for the task: an AIG is a graph, and the quantity to be predicted is structural. Industrial AIGs reach millions of nodes, at which point training a GNN on them exceeds the memory available on a single device and drives training time beyond what is practical, actively preventing the field from scaling to more capable models and more complex tasks.

1.1.3 The Scale Bottleneck

To bypass this bottleneck, the usual responses are to restrict model capacity, adopting a weaker architecture than the task warrants, or to rely on multi-GPU clusters and distributed training. Neither removes the constraint: a single AIG can still exceed the memory of one device, and a distributed setup only makes the problem explicit, since it necessitates splitting the graph.

This ceiling applies to training, not inference. Backpropagation retains the node states, materialised edge messages, and gradients of every layer, whereas a forward-only pass keeps only one layer resident and computes no gradients. Training therefore requires several times the memory inference does, scaling with the number of layers. A circuit too large to train on may consequently still be queried, motivating the question of whether reducing the training input alone suffices.

1.1.4 Limitations of Generic Graph Reduction

Graph reduction, applied as a preprocessing step before training, is standard practice on citation and social graphs, but AIGs differ structurally: an edge encodes a term of the Boolean function the circuit computes, and logic depth, the length of the longest path from primary inputs to primary outputs, correlates with the circuit delay a synthesis pass optimizes for. A cut that balances partition sizes, or a sparsification that preserves the graph’s spectrum, can blindly sever causal cones, obscure logic depth, and break the longest paths, thereby altering the underlying logic function and destroying the structural features accurate prediction depends on.

Merging rather than cutting fails differently. A coarsening that collapses gates into one super-node loses the level separation between them unless its merge criterion reads or understands level, and a criterion that merges gates by logical equivalence carries out part of the optimization a synthesis pass would have carried out, shrinking the very quantity a predictor is asked

to estimate.

1.1.5 Problem Statement and Research Gap

The field scales GNNs on AIGs in two ways today. The first is to train on extracted subcircuits of at most a few thousand gates, a crude sampling that discards global structure and is the de-facto reduction in most circuit-GNN work, including the DeepGate line [Li et al., 2022]. The second is to scale the *architecture*: DeepGate3 [Shi et al., 2024] pools over subcircuits with a transformer, and DeepGate4 [Zheng et al., 2025] uses sparse attention for sub-linear memory. Both change the model, not the input.

Graph reduction shrinks the *input* instead, and is well studied on citation, social and knowledge graphs [Hashemi et al., 2024]. It has never been systematically applied to whole AIGs as a preprocessing step for GNN training, and no work measures which reduction preserves a downstream synthesis label.

The gap this thesis addresses is therefore twofold: (i) there is no systematic evaluation of graph reduction for AIGs, and (ii) there is no evidence on whether AIG-specific adaptation of those reductions is worth its cost.

1.2 Research Questions

1.2.1 RQ1: Task Feasibility and Baseline

How well can a GNN predict the optimizability of a circuit directly from its full, unreduced AIG representation, relative to trivial predictors and published circuit-representation models?

RQ1a: Protocol Sensitivity

How much of the RQ1 result is attributable to the assignment of graphs to splits rather than to the encoder? One model is trained under three protocols differing in nothing but that assignment: a random split, a recipe-disjoint split withholding whole transformation recipes, and the design-disjoint split withholding whole source circuits, the protocol behind every result reported here (3.4.2). The three are nested in what each admits into the test set, and the prediction under test is an ordering of the R^2 of (??),

$$R_{\text{random}}^2 \geq R_{\text{recipe}}^2 \geq R_{\text{design}}^2, \quad (1.1)$$

whose two gaps measure design recognition rather than structural inference (4.1.6).

1.2.2 RQ2: Reduction Efficiency

Applied to AIGs, how do partitioning, sparsification and summarization/coarsening differ in retention η_V, η_E of (??), peak memory, training and inference throughput, and the offline cost of the reduction itself?

1.2.3 RQ3: Predictive Retention and Trade-offs

To what extent do models trained on reduced AIGs retain predictive accuracy (Smooth L1 loss, RMSE, R^2) and circuit-ranking

ability (Spearman correlation) relative to the full-graph baseline of RQ1, and which methods retain the most accuracy per unit of memory and time saved?

RQ3a: Exact and Inexact Compression

How much compression do AIGs admit at zero accuracy cost, and what does each further increment cost once that bound is passed? The provably lossless coarsening of 3.1.5 fixes that bound at its retention ratios η_V and η_E , so every inexact method is quantified by the compression it gains beyond that bound and the accuracy it forfeits.

1.2.4 RQ4: Value of Domain-Informed Adaptation

Do domain-informed reductions, which exploit topological level, reconvergence structure or gate role, retain more predictive accuracy than their generic counterparts at matched retention?

1.2.5 RQ5: Cross-State Inference

Can a model trained exclusively on reduced AIGs, whether partitioned, sparsified or summarized, be queried on full ones at inference time without losing predictive accuracy?

A *structural state* is the form in which a graph reaches the encoder, the reduced $R(G)$ or the unreduced G of 1.2.7, and cross-state inference evaluates weights trained in one state on inputs in the other.

The training and inference costs of 1.1.3 differ by a factor of the encoder depth L . Training on $R(G)$ and querying on G therefore stay inside one memory budget when

$$|\mathcal{V}(G)| + |\mathcal{E}(G)| \leq L (|\mathcal{V}(R(G))| + |\mathcal{E}(R(G))|), \quad (1.2)$$

so any reduction retaining more than a fraction $1/L$ of G admits full-graph inference on the hardware its own training required.

1.2.6 Scope and Delimitations

Optimizability prediction is the evaluation task, not the object of study. The delimitations below were fixed in advance. Limitations found during the work are reported in 5.4.

Target synthesis algorithm. Training and evaluation target one synthesis algorithm, since variance across algorithms would confound the cost attributable to a reduction. The corpus carries three further algorithms, left unused (3.3.3, 6.3.1).

Graph scale. RQ1’s full-graph baseline must fit in memory, so the corpus is bounded at an intermediate rather than an industrial scale (3.4.2, 5.4.2).

Definition of optimizability. Optimizability is relative *node* reduction only. Area, delay and power, the objectives synthesis actually optimizes for, are not modelled (5.4.4).

Reduction families. Partitioning, sparsification and summarization/coarsening are covered, the taxonomy of Hashemi et al. [2024] with partitioning substituted for condensation, which establishes no node correspondence and leaves RQ5 undefined (2.2.3).

Model architecture. One encoder, an edge-aware GCN+ (3.1), held fixed so that outcome differences are attributable to the reduction, which leaves transfer to another encoder untested.

Prediction rather than script generation. Optimizability is predicted, not synthesis scripts: selecting a script would require ranking *algorithms* for one circuit, which a single-algorithm model cannot support (6.3.4).

1.2.7 Problem Formalization

Let G be an AIG with the node roles of (??) and edge inversions of (??) as its features, and let S be a deterministic synthesis algorithm (2.1.3) mapping G to the function-equivalent, optimized $S(G)$. The *optimizability* of G under S is the fraction of nodes the algorithm removes,

$$y(G) = 1 - \frac{|\mathcal{V}(S(G))|}{|\mathcal{V}(G)|} \in [0, 1]. \quad (1.3)$$

Evaluating (1.3) requires executing S , the cost prediction avoids. An encoder f_θ , a GNN in the sense of 2.1.4 with a graph-level readout, is fitted to approximate y and takes values in the same interval $[0, 1]$. A reduction operator R (2.1.5) acts ahead of the encoder, so the argument f_θ receives is G or $R(G)$.

Compression is reported separately as the node and edge retention ratios η_V and η_E of (??). The two are not interchangeable: several methods hold $\eta_V = 1$ while lowering η_E alone. Memory and time differences are taken per graph against the full-graph baseline and carry a 95% bootstrap interval and a two-sided paired rank test, whereas accuracy differences are single-run point estimates (3.4.3).

1.3 Contributions

This thesis reduces the input rather than the model. It makes six contributions. (i) A corpus of AIGs tiered by *optimization state* (whether a target synthesis algorithm has already been applied), extending OpenABC-D [Chowdhury et al., 2021] (3.3). (ii) A systematic benchmark of partitioning, sparsification and summarization as a preprocessing step for GNN training on AIGs, measured on shrinkage, peak memory, speed and predictive retention. (iii) AIG-specific adaptations of each family, built from topological level, reconvergence structure and gate role, compared against their generic counterparts at equal compression. (iv) A coarsening that is provably lossless for this encoder [Bollen et al., 2023], fixing a point of zero accuracy cost against which every lossy method is measured (3.1.5), together with the architectural cost it imposes: normalisation and, where designs run to many levels, node-level positional information (3.2.3). (v) An evaluation of cross-state inference, in which one set of weights

is trained on reduced graphs and queried on full ones. (vi) A controlled comparison of three train/test protocols, random, recipe-disjoint and design-disjoint, ordered by the information each admits into the test set (3.4.2). The hypothesis under test is that the design-disjoint split alone measures structural inference, and that the excess accuracy of the other two is recognition of circuits already seen in training.

The corpus is the enabling contribution: it draws on 55 source designs against OpenABC-D’s 29, and where OpenABC-D varies only the script applied to one starting circuit, it also varies the optimization state that script is applied to. RQ3 and RQ5 depend on that distinction (2.2.1).

1.4 Thesis Outline

Chapter 2 introduces the necessary background (logic synthesis and AIGs, graph neural networks, and the three families of graph reduction) and reviews the literature the gap in 1.1.5 sits in. Chapter 3 describes the encoder and the baselines it is measured against, the reduction methods implemented for each family, the dataset and its labels, and the experimental protocol. Chapter 4 reports results per research question. Chapter 5 interprets them against the literature and states the limitations, and Chapter 6 answers each research question in turn and outlines future work. Command templates, reduction parameters and baseline hyperparameters are recorded in the appendix.

Preliminaries & Related Work

2.1 Preliminaries

(b) The aggregation.

2.1.1 Electronic Design Automation and Logic Synthesis

Electronic Design Automation

(c) The readout.

Position of Logic Synthesis in the Design Flow

Synthesis Scripts, Recipes, and Steps

2.1.2 And-Inverter Graphs

Definition and Structure

Measuring Reduction: Compression Ratio

Node Types and Inverted Edges

Structural Properties: Levels, Logic Cones, Fanout

Structural Distinctions from General Graph Classes

2.1.3 Synthesis Algorithms

Rewriting, Refactoring, Resubstitution, and Balancing

Optimization Objectives: Area, Power, Delay

Synthesis Orchestration

Fixed and Randomized Script Families

2.1.4 Graph Neural Networks

Message Passing Framework

Graph-Level Readout and Pooling

Positional and Structural Encodings

Expressivity and the Weisfeiler-Leman Hierarchy

Training Cost, Memory, and Inference

2.1.5 Graph Reduction: Definitions

Partitioning

Sparsification

Summarization and Coarsening

Exact Summarization

(a) The quotient.

2.2 Related Work

2.2.1 Machine Learning for Logic Synthesis

Quality-of-Result and Optimizability Prediction

Circuit Generation

Learned Synthesis Script Generation

Circuit Representation Learning

Benchmarks and Datasets for ML in EDA

Evaluation Protocols and the Unseen-Design Gap

Models Used as Baselines

SynthNet.

HOGA.

DeepGate4.

2.2.2 Scaling Graph Neural Networks to Large Graphs

Sampling-Based Approaches

Partitioning and Distributed Training

Memory-Efficient Training Techniques

2.2.3 Graph Reduction Techniques

Graph Partitioning

Graph Sparsification

Graph Summarization and Coarsening

Reduction as Preprocessing for GNN Training

2.2.4 Structure-Preserving and Domain-Aware Reduction

Reduction on Directed Acyclic Graphs

Reduction in Circuit and Netlist Domains

2.2.5 Size Generalization and Cross-State Transfer

Size and Distribution Shift in GNNs

Train-Test Structural Mismatch

2.2.6 Research Gap

Methodology

3.1 Architecture

3.1.1 Encoder Formulation

Edge-conditioned messages.

Normalisation, dropout, residual and feed-forward block.

Positional encoding.

Configuration specific to this work.

3.1.2 Partitioned Inputs

3.1.3 Sparsified Inputs

3.1.4 Summarized Inputs

3.1.5 Encoder Variant for Exact Colour Refinement

3.1.6 Baselines for Optimizability Prediction

Trivial Predictors

Random-encoder control.

Adapting Published Circuit Models

Shared porting procedure.

Adaptations required by a graph-level target.

Corrections to the released implementations.

Comparability.

3.2 Graph Reduction

3.2.1 Partitioning

Random Hashing

METIS

Level Slicing

Span-Weighted METIS

3.2.2 Sparsification

Random Edge Dropout

AND-Gate Retention

Random Spanning Forest

PageRank Node Pruning

3.2.3 Summarization and Coarsening

Merge Map and Super-Node Schema

Boundary preservation.

Schema compatibility.

Domain-Specific Coarsening

Graded Colour Refinement

Convolution Matching

Random Within-Type Merging

The Two Roles of Topological Level

3.3 Data

3.3.1 Source Circuits

Structural Characteristics of the Source Designs

3.3.2 Dataset Generation

Random Structural Transformation

Optimization

3.3.3 Tiered Dataset Structure

Tier 0: Base Graphs

Tier 1: Single-Algorithm Optimization

Labelling and Tier Depth

3.3.4 Target Label: Optimizability

Definition

Label Distribution

3.3.5 Graph Representation

Node and Edge Features

Positional Encoding Precomputation

3.3.6 Dataset Analysis

Scale and Structure

Measured Target Distribution

Determinants of the Target

Composition of the Evaluation Splits

3.3.7 Held-Out Hyperparameter Tuning Subset

3.4 Experiments

3.4.1 Research Design

Positive control.

The experiments.

3.4.2 Setup

Data Splitting

Graph Scale Bounds

Dynamic Batching

Constraint on efficiency measurements.

Model Configuration

Hyperparameter Tuning

Training Configuration

Execution Environment and Workload Profile

3.4.3 Evaluation Metrics

Notation.

Evaluation protocol.

Predictive Accuracy

Comparisons derived from these metrics.

Variance already carried by the corpus.

Persisted predictions and stratified re-scoring.

Ranking Quality

Efficiency and Resource Metrics

Memory quantities reported.

Reduction Quality Metrics

Effective receptive field.

Statistical Treatment

Reduction residual.

3.4.4 Reproducibility

Code and Data Availability

Results

4.1 Baseline Predictive Performance on Full AIGs (RQ1)

4.1.1 Comparison Against Naive Baselines

4.1.2 Predictive Accuracy Metrics

4.1.3 Ranking Efficacy

4.1.4 Baseline Model Outcomes

4.1.5 Error Analysis

4.1.6 Protocol Sensitivity: How Much of the Score Is Design Recognition? (RQ1a)

4.1.7 Cost of the Full-Graph Baseline

4.2 Efficiency Profile of Graph Reduction Techniques (RQ2)

4.2.1 Graph Reduction Offline Profile

4.2.2 Training Memory Dynamics and Compute Efficiency

4.2.3 Throughput and Training Speedup

4.2.4 Efficiency Ranking Across Methods

4.3 Predictive Retention and Performance Trade-offs (RQ3)

4.3.1 Accuracy Degradation

4.3.2 Ranking Preservation

4.3.3 Accuracy Loss Attributable to Reduction

4.3.4 The Pareto Front (Trade-Off Analysis)

4.4 Value of Domain-Informed Adaptation (RQ4)

4.4.1 Matched-Compression Pairings

4.4.2 Retention Gap at Matched Compression

4.4.3 Structural Interpretation

4.4.4 Adequacy of the Heuristics Tested

Discussion

5.1 Comparison with Related Work

5.1.1 Positioning Against Optimizability Prediction Work

5.1.2 Positioning Against Graph Reduction Work

5.1.3 Where Direct Comparison Is Not Possible

5.2 Interpretation of Findings

5.2.1 Feasibility of Optimizability Prediction

5.2.2 What Reduction Buys and What It Costs

5.2.3 Why Domain-Informed Adaptation Did or Did Not Help

5.2.4 Cross-State Transfer and Deployment

5.2.5 Unexpected Results

5.3 Practical Implications

5.4 Limitations

5.4.1 Reproducibility

5.4.2 Scalability

5.4.3 Generalizability

5.4.4 Reliability and Validity

Conclusion

6.1 Answers to the Research Questions

6.1.1 RQ1: Task Feasibility and Baseline

6.1.2 RQ2: Reduction Efficiency

6.1.3 RQ3: Predictive Retention and Trade-offs

6.1.4 RQ4: Value of Domain-Informed Adaptation

6.1.5 RQ5: Cross-State Transfer

6.2 Contributions Revisited

6.3 Future Work

6.3.1 Extending Beyond a Single Synthesis Algorithm

6.3.2 Stronger Domain-Aware Reduction

6.3.3 Scaling to Industrial AIGs

6.3.4 From Prediction to Script Generation

Bibliography

- Jeroen Bollen, Jasper Steegmans, Jan Van den Bussche, and Stijn Vansummen. Learning graph neural networks using exact compression. In *Proceedings of the 6th Joint Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)*, pages 8:1–8:9. ACM, 2023. doi: 10.1145/3594778.3594878. Collapses nodes in the same d -step colour-refinement class; provably identical GNN output. Graded variant cr_c with $c = \infty$ (colour refinement) and $c = 1$ (bisimulation).
- Robert K. Brayton and Alan Mishchenko. ABC: An academic industrial-strength verification tool. In *Computer Aided Verification (CAV)*, volume 6174 of *Lecture Notes in Computer Science*, pages 24–40. Springer, 2010. doi: 10.1007/978-3-642-14295-6_5. The citable record for the ABC system. The software itself is at <https://github.com/berkeley-abc/abc>.
- Animesh Basak Chowdhury, Benjamin Tan, Ramesh Karri, and Siddharth Garg. Openabc-d: A large-scale dataset for machine learning guided integrated circuit synthesis. *arXiv preprint arXiv:2110.11292*, 2021. SynthNet. Graph-level labels including percentage of nodes optimized – the closest published analogue of the optimizability label used here.
- Animesh Basak Chowdhury, Shailja Thakur, Hammond Pearce, Ramesh Karri, and Siddharth Garg. Towards the imagenets of ml4eda. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2023. doi: 10.1109/ICCAD57390.2023.10323663. Invited paper. Position piece on benchmark and dataset standards for ML4EDA, motivating the framing used here.
- Mohammad Hashemi, Shengbo Gong, Juntong Ni, Wenqi Fan, B. Aditya Prakash, and Wei Jin. A comprehensive survey on graph reduction: Sparsification, coarsening, and condensation. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, pages 8058–8066, 2024.
- Andreas Kuehlmann, Viresh Paruthi, Florian Krohm, and Malay K. Ganai. Robust boolean reasoning for equivalence checking and functional property verification. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 21(12):1377–1394, 2002. doi: 10.1109/TCAD.2002.804386. Structural hashing (one-level lookup on construction) for AIGs.
- Min Li, Sadaf Khan, Zhengyuan Shi, Naixing Wang, Huang Yu, and Qiang Xu. Deepgate: Learning neural representations of logic gates. In *Proceedings of the Design Automation Conference (DAC)*, 2022.
- Miao Liu, Liwei Ni, Junfeng Liu, Xingyu Meng, Rui Wang, Xiaoze Lin, Xinhua Lai, Xingquan Li, and Jungang Xu. A survey of machine learning approaches in logic synthesis. *ACM Transactions on Design Automation of Electronic Systems*, 31(2):38:1–38:43, 2026. doi: 10.1145/3785362. Divides the field into methods that predict a synthesis outcome and methods that replace the heuristic choosing it.
- Alan Mishchenko and Robert K. Brayton. Scalable logic synthesis using a simple circuit structure. In *Proceedings of the 15th International Workshop on Logic & Synthesis (IWLS)*, pages 15–22, 2006. Not indexed in DBLP; metadata taken from the paper. Source of the resubstitution command and the compress2rs script that C2RS aliases.
- Alan Mishchenko, Satrajit Chatterjee, and Robert K. Brayton. Dag-aware aig rewriting: A fresh look at combinational logic synthesis. In *Proceedings of the 43rd Design Automation Conference (DAC)*, pages 532–535. ACM, 2006. doi: 10.1145/1146909.1147048. Defines the rewrite/refactor primitives and the MFFC as the unit refactoring operates on – the basis for the S6 domain argument.
- Zhengyuan Shi, Ziyang Zheng, Sadaf Khan, Jianyuan Zhong, Min Li, and Qiang Xu. Deepgate3: Towards scalable circuit representation learning. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2024. Pooling transformer over subcircuits. Scales the model, not the input.
- Ziyang Zheng, Shan Huang, Jianyuan Zhong, Zhengyuan Shi, Guohao Dai, Ningyi Xu, and Qiang Xu. Deepgate4: Efficient and effective representation learning for circuit design at scale. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, 2025. URL <https://openreview.net/forum?id=b10lRabU9W>. Sparse graph attention transformer, sub-linear memory, virtual edges over k -hop cones.

First Appendix

A.1 Synthesis Algorithm Configurations

Transformation recipes

Optimization invocations

A.2 Supplementary Figures

A.3 Experimental Configuration

A.4 Hyperparameter Search Space

A.5 Library and Tool Versions

A.6 Reduction Method Parameters

A.7 Baseline Hyperparameters

A.8 Extended Results